

SDET Take-Home Assignment

Complete Project Walkthrough & Interview Explanation Guide

Java 8 • Maven • TestNG • Selenium 4 • Rest Assured 5.4

This document explains every class, every method, the test data, the SQL answers, and how to talk through this project step-by-step in a live interview — as a developer would explain their own code.

Note: The submitted artifact contained only compiled .class files (no .java source). The code shown in this document was reconstructed by analyzing the compiled bytecode (method signatures, field names, embedded string literals, and control-flow shape), so it reflects the real logic and structure, written back out as clean, readable Java.

Table of Contents

1. Project Overview
2. Technology Stack
3. Project Structure
4. Step 1 — `model.CompareResult` (the data holder)
5. Step 2 — `csv.CSVReader` (reading the file)
6. Step 3 — `csv.HeaderComparator` (the core diff logic)
7. Step 4 — `runner.CompareHeaders` (the entry point / main method)
8. Step 5 — `CompareHeadersTest` (unit tests)
9. Step 6 — `api.OrderApiTest` (Rest Assured API tests)
10. Step 7 — `selenium.BaseTest` & `util.FileUtil` (unfinished skeletons)
11. The Test Data — why it was chosen this way
12. SQL Answers Explained
13. How To Run This Project
14. Interview Q&A — likely questions and strong answers
15. Honest Gaps & What I'd Improve

1. Project Overview

This is a take-home assignment designed to test four SDET skills in one project: (1) building comparison/reconciliation logic in plain Java, (2) API testing with Rest Assured, (3) a Selenium UI-testing skeleton, and (4) SQL for data reconciliation.

The centerpiece is a small command-line tool that reads two CSV files (an **expected** file and an **actual** file), compares only their **header rows**, and reports four things: headers only in expected, headers only in actual, headers common to both, and whether the common headers appear in the same relative order in both files.

Around that core, there's a small REST API test suite for an orders endpoint, TestNG unit tests for the comparison logic itself, and a SQL answer sheet for a separate reconciliation problem (comparing two product snapshots).

The one-sentence pitch (say this first in an interview)

"This project is about detecting drift between two data sources — I built the same reconciliation pattern three ways: header-diffing in Java, JOIN-based diffing in SQL, and endpoint-behavior checks in Rest Assured — because that's the core job of an SDET: proving that two things that are supposed to match, actually match."

2. Technology Stack

Technology	Version	Used For
Java	8	Language / language level (maven.compiler.source=1.8)
Maven	-	Build tool, dependency management
TestNG	7.10.2	Test framework — annotations like @Test, @BeforeClass, assertions
Selenium	4.21.0	UI automation (declared, but not implemented in this submission)
Rest Assured	5.4.0	Fluent API testing (given().when().then() style)

pom.xml dependencies (as declared):

```
<dependencies>
  <!-- Selenium -->
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.21.0</version>
  </dependency>

  <!-- TestNG -->
  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>7.10.2</version>
    <scope>test</scope>
  </dependency>

  <!-- Rest Assured -->
  <dependency>
    <groupId>io.rest-assured</groupId>
    <artifactId>rest-assured</artifactId>
    <version>5.4.0</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

3. Project Structure

Standard Maven layout. Package root: **com.sdet.assignment**

```
SDETTakeHome/
├── pom.xml
├── README.md
├── answers.sql
├── AI_TRANSCRIPT.md
├── src/main/java/com/sdet/assignment/
│   ├── model/
│   │   └── CompareResult.java
│   ├── csv/
│   │   ├── CSVReader.java
│   │   └── HeaderComparator.java
│   ├── runner/
│   │   └── CompareHeaders.java
│   ├── api/
│   │   └── OrderApiTest.java
│   ├── selenium/
│   │   └── BaseTest.java
│   └── util/
│       └── FileUtil.java
├── src/test/
│   ├── java/com/sdet/assignment/
│   │   └── CompareHeadersTest.java
│   └── resources/
│       ├── expected_orders.csv
│       └── actual_orders.csv
```

Notice the package-by-layer style (model / csv / runner / api / selenium / util) rather than package-by-feature. That's worth mentioning if asked about design choices — it's a simple, common structure for a small assignment-sized codebase.

4. Step 1 — model.CompareResult

Purpose: A plain POJO that carries the output of a header comparison. No logic here at all — just data plus getters/setters plus a readable toString().

```
package com.sdet.assignment.model;

import java.util.List;

public class CompareResult {

    private List<String> onlyInExpected;
    private List<String> onlyInActual;
    private List<String> commonHeaders;
    private boolean sameRelativeOrder;

    public CompareResult(List<String> onlyInExpected,
                        List<String> onlyInActual,
                        List<String> commonHeaders,
                        boolean sameRelativeOrder) {
        this.onlyInExpected = onlyInExpected;
        this.onlyInActual = onlyInActual;
        this.commonHeaders = commonHeaders;
        this.sameRelativeOrder = sameRelativeOrder;
    }

    public List<String> getOnlyInExpected() { return onlyInExpected; }
    public void setOnlyInExpected(List<String> onlyInExpected) {
        this.onlyInExpected = onlyInExpected;
    }

    public List<String> getOnlyInActual() { return onlyInActual; }
    public void setOnlyInActual(List<String> onlyInActual) {
        this.onlyInActual = onlyInActual;
    }

    public List<String> getCommonHeaders() { return commonHeaders; }
    public void setCommonHeaders(List<String> commonHeaders) {
        this.commonHeaders = commonHeaders;
    }

    public boolean isSameRelativeOrder() { return sameRelativeOrder; }
    public void setSameRelativeOrder(boolean sameRelativeOrder) {
        this.sameRelativeOrder = sameRelativeOrder;
    }

    @Override
    public String toString() {
        return "CompareResult{onlyInExpected=" + onlyInExpected
            + ", onlyInActual=" + onlyInActual
            + ", commonHeaders=" + commonHeaders
            + ", sameRelativeOrder=" + sameRelativeOrder + '}';
    }
}
```

Confirmed from the compiled class: field names (onlyInExpected, onlyInActual, commonHeaders, sameRelativeOrder), the 4-argument constructor signature, every getter/setter pair, and the literal toString() prefix strings all appear directly in the bytecode's string table.

How to explain it in an interview

"I kept this as a dumb data container on purpose — no logic in the model. All the actual comparison decisions live in HeaderComparator, so this class is trivially testable and has a single responsibility: hold a result."

5. Step 2 — csv.CSVReader

Purpose: Open a CSV file and read *only the header row* (line 1), split it into a list of trimmed header names.

```
package com.sdet.assignment.csv;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

public class CSVReader {

    public List<String> readHeaders(String filePath) throws IOException {
        List<String> headers = new ArrayList<>();

        try (BufferedReader reader = new BufferedReader(new FileReader(filePath))) {
            String firstLine = reader.readLine();

            if (firstLine == null || firstLine.trim().isEmpty()) {
                throw new IOException("CSV file is empty or header row is missing.");
            }

            String[] headerArray = firstLine.split(",");

            for (String header : headerArray) {
                String trimmedHeader = header.trim();
                if (!trimmedHeader.isEmpty()) {
                    headers.add(trimmedHeader);
                }
            }

            if (headers.isEmpty()) {
                throw new IOException("Header row contains no valid headers.");
            }
        }

        return headers;
    }
}
```

Confirmed from bytecode: try-with-resources usage (the compiler emits an `addSuppressed()` call, which only appears for try-with-resources blocks), the two exact error message strings, and the `readLine()` -> `split(",")` -> `trim()` sequence.

Edge cases it handles

- Empty file, or a file whose very first line is blank
- A header line that's just commas / whitespace (no real header names survive trimming)
- Trailing/leading whitespace around individual header names

Edge cases it does NOT handle (good interview talking point)

- Quoted CSV fields containing a comma, e.g. "Smith, John" — naive `split(",")` would break this apart incorrectly
- Different delimiters (semicolon-delimited CSVs, TSVs)
- BOM (byte-order-mark) characters some Excel-exported CSVs prepend to the first header

6. Step 3 — csv.HeaderComparator (the core logic)

Purpose: Given two header lists, compute the four-part diff. This is the class an interviewer will drill into the most.

```
package com.sdet.assignment.csv;

import com.sdet.assignment.model.CompareResult;
import java.util.ArrayList;
import java.util.List;

public class HeaderComparator {

    public CompareResult compareHeaders(List<String> expectedHeaders,
                                       List<String> actualHeaders) {

        List<String> onlyInExpected = new ArrayList<>();
        List<String> onlyInActual = new ArrayList<>();
        List<String> commonHeaders = new ArrayList<>();

        // headers present in expected but missing from actual
        for (String header : expectedHeaders) {
            if (!actualHeaders.contains(header)) {
                onlyInExpected.add(header);
            } else {
                commonHeaders.add(header);
            }
        }

        // headers present in actual but missing from expected
        for (String header : actualHeaders) {
            if (!expectedHeaders.contains(header)) {
                onlyInActual.add(header);
            }
        }

        // do the common headers appear in the same relative order in both lists?
        boolean sameRelativeOrder = true;
        int lastIndex = -1;

        for (String header : commonHeaders) {
            int currentIndex = actualHeaders.indexOf(header);
            if (currentIndex < lastIndex) {
                sameRelativeOrder = false;
                break;
            }
            lastIndex = currentIndex;
        }

        CompareResult result = new CompareResult(
            onlyInExpected, onlyInActual, commonHeaders, sameRelativeOrder);

        return result;
    }
}
```

Confirmed from bytecode: two separate iterator loops (one driven by expected, one by actual) each calling `List.contains()`, an `indexOf()` call inside a third loop, and locals literally named `lastIndex` and `currentIndex` — which is the compiler preserving debug-symbol names from the original source.

Walk through the order-check algorithm out loud (this is the good part)

Track the highest actual-list index seen so far (`lastIndex`). Walk through `commonHeaders` in the order they appear in expected. For each header, find where it sits in `actualHeaders`. If that position is *earlier* than the last one we saw, a header has moved backwards — order is broken, stop immediately. Otherwise, update `lastIndex` and keep going. This is an $O(n)$ single-pass check for 'is this a subsequence in the same relative order,' assuming duplicates aren't a concern.

Complexity — be ready for this question

`List.contains()` and `indexOf()` on a List are $O(n)$ each, and they're called inside loops, so the whole comparator is roughly $O(n^2)$ in the number of headers. For a header row (a handful of columns) that's completely fine. I'd mention: 'If this needed to scale to hundreds of columns, I'd swap the Lists for a `LinkedHashSet` or a `HashMap<String,Integer>` of header-to-index to get $O(1)$ lookups and bring it down to $O(n)$.'

7. Step 4 — runner.CompareHeaders (the entry point)

Purpose: The class with `public static void main()` — this is what you actually run. Wires the reader and comparator together and prints a human-readable report.

```
package com.sdet.assignment.runner;

import com.sdet.assignment.csv.CSVReader;
import com.sdet.assignment.csv.HeaderComparator;
import com.sdet.assignment.model.CompareResult;
import java.util.List;

public class CompareHeaders {

    public static void main(String[] args) {

        String expectedFile = "src/test/resources/expected_orders.csv";
        String actualFile   = "src/test/resources/actual_orders.csv";

        try {
            CSVReader csvReader = new CSVReader();
            HeaderComparator comparator = new HeaderComparator();

            List<String> expectedHeaders = csvReader.readHeaders(expectedFile);
            List<String> actualHeaders   = csvReader.readHeaders(actualFile);

            CompareResult result = comparator.compareHeaders(expectedHeaders, actualHeaders);

            System.out.println("=====");
            System.out.println("CSV HEADER COMPARISON RESULT");
            System.out.println("=====");

            System.out.println("Only in Expected CSV:");
            for (String header : result.getOnlyInExpected()) {
                System.out.println(header);
            }

            System.out.println("Only in Actual CSV:");
            for (String header : result.getOnlyInActual()) {
                System.out.println(header);
            }

            System.out.println("Common Headers:");
            for (String header : result.getCommonHeaders()) {
                System.out.println(header);
            }

            System.out.println("Common Headers in Same Relative Order: "
                               + result.isSameRelativeOrder());

        } catch (java.io.IOException e) {
            System.out.println("Error: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("Unexpected Error: " + e.getMessage());
        }
    }
}
```

*Confirmed from bytecode: the exact hardcoded file paths, the '=====
divider string, each of the four section header strings printed in order, and two separate catch blocks — one for*

IOException specifically, one generic Exception fallback.

How to explain it in an interview

"This is deliberately thin — it has one job, orchestration. It doesn't contain any comparison logic itself, it just calls CSVReader then HeaderComparator and formats the output. If I were productionizing this I'd replace the hardcoded paths with args[0] / args[1], and I'd probably return a proper exit code (0 for match, non-zero for drift) so it could gate a CI pipeline."

8. Step 5 — CompareHeadersTest (unit tests)

Purpose: TestNG unit tests that exercise HeaderComparator directly, in isolation from file I/O. Five scenarios, using a fixed sample header set: order_id, customer_id, amount, currency, status.

```
package com.sdet.assignment;

import com.sdet.assignment.csv.HeaderComparator;
import com.sdet.assignment.model.CompareResult;
import org.testng.Assert;
import org.testng.annotations.Test;
import java.util.Arrays;
import java.util.List;

public class CompareHeadersTest {

    private HeaderComparator comparator = new HeaderComparator();

    @Test
    public void testIdenticalHeaders() {
        List<String> expected = Arrays.asList(
            "order_id", "customer_id", "amount", "currency", "status");
        List<String> actual = Arrays.asList(
            "order_id", "customer_id", "amount", "currency", "status");

        CompareResult result = comparator.compareHeaders(expected, actual);

        Assert.assertTrue(result.getOnlyInExpected().isEmpty());
        Assert.assertTrue(result.getOnlyInActual().isEmpty());
        Assert.assertEquals(result.getCommonHeaders().size(), 5);
        Assert.assertTrue(result.isSameRelativeOrder());
    }

    @Test
    public void testMissingHeader() {
        // actual is missing "status" -> should show up in onlyInExpected
        List<String> expected = Arrays.asList(
            "order_id", "customer_id", "amount", "currency", "status");
        List<String> actual = Arrays.asList(
            "order_id", "customer_id", "amount", "currency");

        CompareResult result = comparator.compareHeaders(expected, actual);

        Assert.assertTrue(result.getOnlyInExpected().contains("status"));
    }

    @Test
    public void testDifferentOrder() {
        // same headers, different order -> sameRelativeOrder should be false
        List<String> expected = Arrays.asList(
            "order_id", "customer_id", "amount", "currency", "status");
        List<String> actual = Arrays.asList(
            "customer_id", "order_id", "amount", "currency", "status");

        CompareResult result = comparator.compareHeaders(expected, actual);

        Assert.assertFalse(result.isSameRelativeOrder());
    }

    @Test
    public void testExtraHeader() {
```

```

// actual has an extra column not in expected -> onlyInActual
List<String> expected = Arrays.asList(
    "order_id", "customer_id", "amount", "currency", "status");
List<String> actual = Arrays.asList(
    "order_id", "customer_id", "amount", "currency", "status", "discount");

CompareResult result = comparator.compareHeaders(expected, actual);

Assert.assertTrue(result.getOnlyInActual().contains("discount"));
}

@Test
public void testEmptyHeaders() {
    List<String> expected = Arrays.asList();
    List<String> actual = Arrays.asList();

    CompareResult result = comparator.compareHeaders(expected, actual);

    Assert.assertTrue(result.getCommonHeaders().isEmpty());
    Assert.assertTrue(result.isSameRelativeOrder());
}
}

```

Confirmed from bytecode: five @Test methods with these exact names, the literal header strings (order_id, customer_id, amount, currency, status), calls to Assert.assertTrue / assertFalse / assertEquals, and Arrays.asList usage. The exact assertion bodies for testMissingHeader / testExtraHeader / testEmptyHeaders are reconstructed logically from the method name and the assertions available in the constant pool — the shape shown here is the standard, most natural way to write each of these cases.

Why this class matters in an interview

"I unit-tested the comparator by itself, with in-memory lists, instead of only testing it through the CSV files. That means these tests run in milliseconds, don't touch the filesystem, and pin down each behavior — identical, missing, reordered, extra, and empty — as its own readable test case."

9. Step 6 — api.OrderApiTest (Rest Assured)

Purpose: API tests for an orders endpoint, hitting `https://api.example.com`, covering positive, negative, boundary, and non-functional (performance + content-type) cases.

```
package com.sdet.assignment.api;

import io.restassured.RestAssured;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.Test;

import static io.restassured.RestAssured.given;
import static org.hamcrest.Matchers.equalTo;
import static org.hamcrest.Matchers.lessThan;

public class OrderApiTest {

    @BeforeClass
    public void setup() {
        RestAssured.baseURI = "https://api.example.com";
    }

    @Test
    public void verifyValidOrder() {
        given()
            .when()
                .get("/api/orders/ORD-1001")
            .then()
                .statusCode(200)
                .body("status", equalTo("PAID"));
    }

    @Test
    public void verifyInvalidOrder() {
        given()
            .when()
                .get("/api/orders/INVALID")
            .then()
                .statusCode(404); // order does not exist
    }

    @Test
    public void verifyEmptyOrderId() {
        given()
            .when()
                .get("/api/orders/")
            .then()
                .statusCode(400); // missing path parameter
    }

    @Test
    public void verifyResponseTime() {
        given()
            .when()
                .get("/api/orders/ORD-1001")
            .then()
                .time(lessThan(2000L)); // milliseconds
    }

    @Test
    public void verifyContentType() {
```

```

        given()
        .when()
            .get("/api/orders/ORD-1001")
        .then()
            .contentType("application/json");
    }
}

```

Confirmed from bytecode: baseURI literal, all five @Test method names, the exact paths /api/orders/ORD-1001, /api/orders/INVALID and /api/orders/ (trailing slash, no ID), the body("status", equalTo("PAID")) assertion, a Long-valued lessThan(...) matcher passed to time(...), and contentType("application/json"). The exact expected status codes for the invalid/empty cases (404 / 400) aren't printed as literals in the bytecode — those are the conventional REST status codes for 'not found' and 'bad request', shown here as the most natural implementation; the assignment's actual number may differ, so verify against the live API when running.

How to explain the design in an interview

- **verifyValidOrder** — the happy path: known-good order ID returns 200 and the right status field
- **verifyInvalidOrder** — negative test: an ID that doesn't exist
- **verifyEmptyOrderId** — boundary test: no ID supplied at all
- **verifyResponseTime** — non-functional/performance assertion baked right into the fluent chain
- **verifyContentType** — a contract check that's easy to forget but easy to break (e.g. someone returns XML or text/plain by accident)

"I like that Rest Assured lets the given / when / then structure map directly onto Arrange / Act / Assert — it reads like a sentence, which makes these easy for a non-Java reader to review too."

10. Step 7 — selenium.BaseTest & util.FileUtil (skeletons)

Honest finding: When I inspected the compiled bytecode for these two classes, neither one contains any real method bodies beyond the default constructor. They compile down to empty shells — essentially placeholders.

```
package com.sdet.assignment.selenium;

public class BaseTest {
    // Intended to hold shared WebDriver setup/teardown
    // (e.g. @BeforeMethod driver init, @AfterMethod driver.quit())
    // for any future Selenium test classes to extend.
    // Not implemented in this submission.
}

package com.sdet.assignment.util;

public class FileUtil {
    // Intended to hold shared file-handling helpers
    // (e.g. path validation, file-existence checks)
    // reusable across CSVReader and other I/O classes.
    // Not implemented in this submission.
}
```

How to handle this if an interviewer notices

Don't get defensive — own it plainly: "Selenium is in the pom.xml because the assignment mentions UI automation as part of the stack, and I scaffolded a BaseTest for shared WebDriver setup, but I didn't have a UI in scope to test against, so I focused my time on the CSV comparator, the API tests, and the SQL. FileUtil was meant to hold shared file-handling logic pulled out of CSVReader, but the project stayed small enough that I never needed to factor it out." This reads as self-aware prioritization, not a mistake.

11. The Test Data — why it was chosen this way

expected_orders.csv

```
order_id,customer_id,amount,currency,status,created_at,country
1,C001,100.50,GBP,PAID,2026-05-20,UK
2,C002,40.00,GBP,PENDING,2026-05-20,UK
```

actual_orders.csv

```
order_id,customer_id,total_amount,currency,status,processed_at,country_code
1,C001,100.50,GBP,PAID,2026-05-20,GB
2,C002,40.00,GBP,PENDING,2026-05-20,GB
```

What's actually different, header by header

Expected header	Actual header	What changed
amount	total_amount	Renamed column
created_at	processed_at	Renamed column
country	country_code	Renamed column + data changed (UK -> GB)
order_id, customer_id, currency, status	same	Unchanged, common headers

So running `CompareHeaders.main()` against this pair reports: 3 headers 'only in expected' (amount, created_at, country), 3 headers 'only in actual' (total_amount, processed_at, country_code), 4 common headers (order_id, customer_id, currency, status), and since none of them moved position relative to each other, `sameRelativeOrder` is true.

Why this is a smart test fixture: it simulates the single most common real-world SDET scenario — two systems (or two versions of the same system) that describe the same underlying orders but disagree on column naming and on how a value is encoded (UK vs the ISO country code GB). It's realistic without being complicated.

12. SQL Answers Explained

The SQL half reconciles two snapshots of a products table — `previous_products` and `current_products` — the same 'diff two data sources' theme as the CSV comparator, just expressed with joins instead of Java lists.

Question 1 — Price Changes

```
SELECT
    p.product_id,
    p.product_name,
    p.price AS old_price,
    s.price AS new_price
FROM previous_products p
INNER JOIN current_products s
ON p.product_id = s.product_id
WHERE p.price <> s.price;
```

Logic: INNER JOIN keeps only products that exist in *both* snapshots (matched by `product_id`), then the WHERE clause filters down to rows where the price differs between the two. Products that only exist in one snapshot are excluded entirely by the INNER JOIN — that's intentional, because 'price changed' only makes sense for a product that existed both before and after.

Question 2 — New Products

```
SELECT
    c.product_id,
    c.product_name,
    c.price
FROM current_products c
LEFT JOIN previous_products p
ON c.product_id = p.product_id
WHERE p.product_id IS NULL;
```

Logic: LEFT JOIN from `current_products` keeps every current row, and attaches a matching previous row if one exists. Where there's no match, all of `previous_products`' columns come back NULL — so filtering on `p.product_id IS NULL` isolates exactly the products that appear now but didn't exist in the previous snapshot: brand-new products.

Question 3 — Missing Products

```
SELECT
    p.product_id,
    p.product_name,
    p.price
FROM previous_products p
LEFT JOIN current_products c
ON p.product_id = c.product_id
WHERE c.product_id IS NULL;
```

Logic: The exact mirror image of Question 2 — LEFT JOIN starting from `previous_products` this time, so this finds products that existed before but have since been removed/discontinued.

Question 4 — Status Changes

```
SELECT
    p.product_id,
    p.product_name,
    p.status AS old_status,
    c.status AS new_status
FROM previous_products p
INNER JOIN current_products c
ON p.product_id = c.product_id
WHERE p.status <> c.status;
```

Logic: Same pattern as Question 1, just swapping the compared column from price to status — e.g. catching a product that flipped from 'active' to 'discontinued' between snapshots.

The pattern to name out loud in an interview

"All four queries are the same two building blocks: INNER JOIN + WHERE <> for 'this attribute changed on a record that exists in both,' and LEFT JOIN + IS NULL for 'this record only exists on one side.' Once you see that pattern, you can answer almost any reconciliation question between two tables with the same shape."

A gotcha worth mentioning proactively

`p.price <> s.price` will silently exclude a row where either price is NULL, because SQL's three-valued logic makes NULL <> anything evaluate to NULL/unknown, not true. If NULL prices are possible, the safer version is `p.price IS DISTINCT FROM s.price` (Postgres) or an explicit NULL-aware comparison in dialects that don't support IS DISTINCT FROM.

13. How To Run This Project

Run the CSV comparator

```
mvn compile
mvn exec:java -Dexec.mainClass="com.sdet.assignment.runner.CompareHeaders"
```

or, after packaging:

```
mvn package
java -cp target/classes com.sdet.assignment.runner.CompareHeaders
```

Run all TestNG tests (unit tests + API tests)

```
mvn test
```

This runs both CompareHeadersTest (fast, in-memory, no network) and OrderApiTest (requires the API base URL to be reachable — in a real run this would point at a live or mocked orders service rather than the placeholder api.example.com).

14. Interview Q&A — Likely Questions and Strong Answers

Q: Walk me through this project.

A: It's a small SDET assignment with four parts that all share one idea — proving two things that should match actually do. A Java CSV header comparator with its own unit tests, a Rest Assured API test suite for an orders endpoint, SQL reconciliation queries between two product snapshots, and a scaffolded (but unused) Selenium base class.

Q: Why compare only headers and not the actual row data?

A: The assignment scoped it to header/schema drift specifically — that's usually the first thing you check before you even trust row-level data, because if the columns don't line up, comparing values is meaningless or actively misleading. I'd frame row-data comparison as the natural next iteration.

Q: How would you extend HeaderComponent to also compare row values, not just headers?

A: I'd read each CSV fully into a List of Maps (or a small Row object) keyed by header, then join rows between the two files on a natural key like order_id, and for each shared column, compare values and collect a list of {row key, column, expected value, actual value} mismatches — the same reconciliation shape as the SQL Question 1 query, just done in Java.

Q: Your comparator uses List.contains() and indexOf() in loops — is that efficient?

A: For a header row it's totally fine — a handful of columns. It is $O(n^2)$ in theory since each contains()/indexOf() call is a linear scan. If it needed to scale, I'd back it with a HashMap mapping header to its index for $O(1)$ lookups.

Q: What happens if a CSV has duplicate headers?

A: The current logic doesn't explicitly guard against duplicates — contains() only tells you 'present or not', so a duplicate header would just get skipped after the first match rather than flagged. If duplicate detection mattered, I'd tally header counts and diff the counts, not just set-membership.

Q: Why did you choose TestNG over JUnit?

A: The assignment specified TestNG in the tech stack, and it's a common enterprise choice — its built-in support for test groups, parallel execution, and data providers tends to matter more once a suite grows, and it plays nicely with Rest Assured and Selenium in the same project.

Q: Why is BaseTest empty — didn't you need Selenium?

A: There wasn't a UI in scope for this particular assignment, so rather than write a Selenium test against nothing, I scaffolded the base class to show where shared WebDriver setup/teardown would live, and put my effort into the parts that had real functionality to test.

Q: What would you add if you had one more day?

A: Row-level CSV data comparison (not just headers), CLI args instead of hardcoded file paths in CompareHeaders, a real Selenium test behind BaseTest, and negative/edge-case tests for CSVReader itself (malformed rows, wrong delimiter, BOM characters).

Q: How does the SQL section relate to the Java section?

A: Same underlying skill, two tools. The Java comparator diffs header lists; the SQL queries diff two table snapshots using INNER JOIN + WHERE <> for 'changed' and LEFT JOIN + IS NULL for 'only on one side.'

I'd point out that pattern is basically the SQL equivalent of `onlyInExpected` / `onlyInActual` / `commonHeaders` in `HeaderComparator`.

15. Honest Gaps & What I'd Improve

- Hardcoded file paths in `CompareHeaders.main()` instead of accepting them as CLI arguments.
- `CSVReader.split(",")` is naive — breaks on quoted fields that contain commas; a real CSV parser (e.g. Apache Commons CSV or OpenCSV) would be safer for production data.
- Only header-level comparison exists; there's no row-level / cell-level data comparison yet.
- `BaseTest` (Selenium) and `FileUtil` are unimplemented placeholders.
- `OrderApiTest` points at a placeholder base URL (`https://api.example.com`) rather than a real or mocked endpoint, so it can't actually execute successfully as-is.
- No logging framework — everything goes to `System.out`; fine for a CLI demo, not ideal for anything long-running.
- SQL's `<>` comparisons would silently miss NULL-vs-value differences; `IS DISTINCT FROM` (or the dialect equivalent) is the safer choice if NULLs are possible.

Reminder: this document was produced by decompiling/analyzing the compiled .class files in the submitted jar (no .java sources were included). Method names, field names, string literals, and control-flow shape are taken directly from the bytecode; a few implementation details called out above (e.g. exact HTTP status codes, the precise body of a couple of unit test assertions) were filled in using the most conventional, idiomatic choice for that signature — verify those specific spots against the live system if it matters for your interview.